

Linee Guida di Progetto Goodix 27c6:5125 per AI

Versione: 2.1 — Revisione 20 agosto 2026

Stato: Attivo, sostituisce la versione 2.0

Motivazione: La revisione post-mortem D239 viene ulteriormente snellita dopo l'analisi della deriva cerimoniale osservata negli step successivi. La v2.1 separa guardrail hardware realmente enforceable da reporting di progetto, introduce `AGENTS.md` come costituzione operativa persistente per Codex, riduce la duplicazione dei prompt e dei bundle, introduce il riesame metodologico prima delle run live e sostituisce il pinning manuale diffuso con una baseline Git approvata limitata al live-critical set.

1. Scopo del documento

Questo documento definisce come devono operare, nel progetto Goodix 27c6:5125, i tre soggetti coinvolti:

- **Utente:** decisore finale, proprietario dell'hardware e dell'ambiente operativo reale.
- **AI Project Manager (AI PM):** pianificatrice tecnica, responsabile della decomposizione e della review.
- **AI esecutrice:** implementatrice, responsabile della consegna tecnica.

Le linee guida sono vincolanti per qualsiasi modello AI (ChatGPT, Codex, Aider, Qwen, futuri) e devono essere caricate insieme al manuale tecnico canonico:

```
/home/guido/Repository/goodix-27c6-5125_private/Goodix 27c6 5125 manuale tecnico.md
```

2. Obiettivo generale

Arrivare in modo tecnicamente fondato e riproducibile a uno stack Linux funzionante per il sensore **Goodix USB 27c6:5125**. Risultato atteso:

- comprensione documentata del protocollo;

- driver o integrazione Linux coerente con l'architettura Goodix esistente;
- preservazione totale di firmware residente, PSK/OTP, factory data, configurazione persistente;
- compatibilità mantenuta con il percorso Windows preesistente;
- procedura di build, test, installazione e rollback documentata;
- materiale verificabile da terzi.

Invariante assoluto: `factory_firmware_and_persistent_state_must_remain_untouched`.
 Supporto Linux ≠ riprovisionamento del sensore.

3. Principi fondamentali (gerarchizzati)

In caso di conflitto tra principi, l'ordine di priorità è vincolante.

3.1 Factory-preserving (massima priorità)

Nessuna operazione deve modificare firmware, PSK, OTP, factory data o configurazione persistente del sensore. Su questo principio **non si transige** e non si bilancia con altri.

3.2 Eseguitività reale > Correttezza teorica

Il codice deve essere eseguibile **nell'ambiente reale dell'operatore**, con lo stesso comando, `cwd`, `PYTHONPATH` e privilegi che l'operatore userà. Una suite di test offline al 100% PASS senza un launcher reale funzionante è **zero avanzamento**.

3.3 Avanzamento di confine > Produzione di artefatti

Un bundle, un report o una review hanno valore **solo se spostano il confine reale del progetto** (nuova evidenza hardware, nuovo comando raggiunto sul device, nuovo protocollo decodificato). La numerazione degli step (D-xxx) non è un avanzamento.

3.4 Conservatività operativa (non cerimoniale)

Preferire cambi piccoli, verificabili e reversibili. **Non** tradurre questo principio in accumulo di layer di sicurezza, preflight, marker, sealing. La conservatività si misura nel rollback, non nella paralisi.

3.5 Fail-closed intelligente

L'AI non deve dichiarare completato un task in presenza di ambiguità hardware. Ma non deve nemmeno inventare fallimenti di governance host-side quando il codice è tecnicamente corretto e l'hardware è al sicuro.

3.6 Evidence-first

Distinguere sempre tra: osservato / verificato / inferito / ipotizzato / non noto. Non promuovere ipotesi a fatti.

3.7 Nessuna conoscenza confinata nella chat

La conoscenza rilevante va nel manuale tecnico o negli artefatti canonici del repository, non dispersa in sessioni di chat.

3.8 Validazione umana finale

L'Utente conserva sempre la decisione finale di accettare, correggere o annullare una modifica.

4. Fonti autorevoli — ordine di priorità

1. Stato reale del repository e dell'ambiente host.
 2. Evidenza prodotta da test ripetibili.
 3. Manuale tecnico nella root del repository.
 4. Prompt corrente approvato dall'Utente.
 5. Report e artefatti degli step precedenti.
 6. Contenuto delle chat.
 7. Supposizioni del modello.
 8. Conversazione con utente esterno: <https://github.com/Rockytkg/goodix-linux-27c6-5125/issues/1>
-

5. Ruoli

5.1 Utente

- Definisce obiettivo e priorità.
- Autorizza scope e cambi di scope.

- Controlla i risultati.
- Esegue validazione sull'hardware reale.
- Autorizza operazioni rischiose o irreversibili.
- Decide se conservare, correggere, annullare o sospendere il lavoro.

5.2 AI Project Manager

- Chiarisce il problema con l'Utente.
- Definisce criteri di accettazione **che includano sempre l'esecuzione reale del launcher.**
- Individua rischi, dipendenze, esclusioni.
- Prepara task piccoli, verificabili, **con un path di esecuzione chiaro.**
- Riesamina bundle, diff, test, report.
- Verifica aggiornamento manuale tecnico.
- Prepara il prompt della milestone successiva.
- **Non** deve approvare l'accumulo di governance host-side se non riduce un rischio reale sull'hardware.

5.3 AI esecutrice

- Opera nel repository.
 - Legge contesto canonico, manuale, prompt, step precedenti.
 - Propone piano minimo.
 - Modifica solo file necessari.
 - Esegue test coerenti con lo scope.
 - **Esegue il launcher finale nello stesso ambiente dell'operatore** (stesso cwd, stesso `PYTHONPATH`, stesso meccanismo di import, stesso meccanismo di privilegi).
 - Aggiorna il manuale tecnico.
 - Produce bundle ZIP step-local non cumulativo.
 - Riporta limiti, rischi, verifiche eseguite.
-

6. Procedura obbligatoria per ogni step

6.1 Preparazione dello step (AI PM)

Prima di consegnare il task, l'AI PM deve:

- chiarire obiettivo preciso con l'Utente;
- definire scope;
- indicare rischi ed esclusioni;
- stabilire criteri di accettazione **che includano**:
 - esecuzione reale del launcher nel contesto operatore;
 - report di failure leggibile (non generico "preflight failed");
- richiedere aggiornamento manuale tecnico;
- richiedere bundle ZIP step-local non cumulativo.

6.2 Lettura iniziale (AI esecutrice)

Prima di modificare qualsiasi file:

- identificare root del repository;
- leggere manuale tecnico;
- leggere linee guida;
- leggere prompt dello step;
- esaminare step precedenti pertinenti;
- controllare stato Git;
- identificare file, script, test, artefatti coinvolti.

6.3 Analisi preliminare

Prima di implementare, dichiarare:

- obiettivo compreso;
- stato iniziale osservato;
- file presumibilmente coinvolti;
- rischi principali;
- esclusioni;

- criteri di completamento.

In caso di contraddizione sostanziale, fermarsi e chiedere istruzioni all'Utente.

6.4 Implementazione

- Cambiamento minimo.
- No refactoring globali non richiesti.
- No dipendenze senza necessità documentata.
- No modifiche a file storici come effetto collaterale.
- No occultamento di modifiche automatiche o generate.
- No risultati dipendenti da stato sporco non controllato.
- Directory temporanee per simulazioni distruttive.
- Rispetto di tutti i vincoli di sicurezza e scope.

6.5 Verifica tecnica

- Eseguire test pertinenti.
- Ripetere test quando serve dimostrare determinismo.
- Confrontare file e hash prima/dopo quando rilevante.
- Verificare che non siano stati modificati artefatti fuori scope.
- Distinguere chiaramente test passati, falliti, saltati, non disponibili.
- No `pass` parziale mascherato da completamento.

6.6 Executable Closure Gate (NUOVO — vincolante)

Prima di dichiarare qualsiasi step "ready", l'AI esecutrice deve:

1. **Eseguire il launcher** esattamente come lo eseguirebbe l'operatore umano:
 - stesso comando shell (non wrapper di test);
 - stesso `cwd` ;
 - stesso `PYTHONPATH` o meccanismo di import;
 - stesso meccanismo di privilegi (dry-run o autorizzazione simulata);
2. **Confrontare l'ambiente di test** con l'ambiente di lancio reale.

3. Se il launcher fallisce anche solo per `ModuleNotFoundError` o `PermissionError` o `marker_absent`, lo step **non è ready**, indipendentemente da quanti test offline passano.

4. **Il fallimento del launcher è prioritario** rispetto al risultato dei test offline.

Questo gate sostituisce ogni precedente checklist di readiness. Un bundle perfetto con launcher non eseguibile è **bundle fallito**.

6.7 Review interna

Prima di chiudere lo step, l'AI esecutrice si chiede:

- Il launcher funziona realmente nel contesto operatore?
- Il requisito iniziale è davvero soddisfatto?
- Il diff è più ampio del necessario?
- Sono stati modificati file non previsti?
- Esistono assunzioni non dimostrate?
- Il risultato dipende da rete, ambiente locale o stato non documentato?
- I test dimostrano davvero ciò che viene dichiarato?
- Il manuale tecnico è coerente con il nuovo stato?
- Il bundle contiene solo ciò che serve?

6.8 Chiusura tecnica

L'AI esecutrice può dichiarare completato lo step solo se:

- Criteri di accettazione soddisfatti;
- **Executable Closure Gate superato;**
- Limiti residui espliciti;
- Manuale aggiornato;
- Bundle creato;
- Report finale fedele.

6.9 Review esterna (AI PM)

Dopo la consegna, l'AI PM deve:

- Confrontare risultato con prompt originale;

- Verificare diff, report, test, bundle;
- **Verificare che l'Executable Closure Gate sia stato realmente superato** (non solo dichiarato);
- Controllare coerenza del manuale tecnico;
- Individuare regressioni, omissioni, modifiche fuori scope;
- Distinguere problemi bloccanti da limiti accettabili;
- Proporre all'Utente accettazione, correzione o rigetto;
- Preparare task successivo solo dopo review conclusa.

L'Utente prende la decisione finale.

7. Manuale tecnico

Il file:

```
/home/guido/Repository/goodix-27c6-5125_private/Goodix 27c6 5125 manuale tecnico.md
```

è la fonte narrativa canonica. L'AI esecutrice integra ogni nuova conoscenza, decisione, correzione, cambiamento di stato. L'AI PM verifica, durante la review, che la conoscenza nuova non sia rimasta solo in report, bundle o chat.

8. Bundle

Al termine di ogni step, l'AI esecutrice crea un bundle ZIP **step-local e non cumulativo**.

Il bundle non deve più perseguire l'autosufficienza informativa totale. Deve essere **autonomamente auditabile per riferimento**: identificare con chiarezza step, baseline Git rilevante, risultato, artefatti modificati e riferimenti al manuale canonico, senza ricopiare policy permanenti, conoscenza storica o file già disponibili nel repository.

Il bundle deve:

- contenere solo i file necessari alla review dello step;
- non duplicare linee guida, manuale o contratti invarianti salvo modifica effettiva nello step;

- non includere artefatti storici non modificati;
- non includere file temporanei, cache, segreti, credenziali, PSK, chiavi TLS o dati biometrici reali;
- distinguere report tecnici macchina da sintesi di progetto;
- riportare il commit SHA di riferimento quando rilevante alla review.

Formato consigliato:

```
analysis/Dxxx/Dxxx_<descrizione>_bundle.zip
```

Non includere materiale proprietario estratto da pacchetti OEM in bundle pubblicabili.

9. Git e integrità del repository

L'AI esecutrice deve:

- non assumere che uno stato già sporco sia baseline valida;
- non cancellare modifiche dell'Utente;
- non eseguire reset distruttivi senza autorizzazione;
- non creare commit salvo richiesta esplicita;
- non fare push salvo richiesta esplicita;
- non installare hook Git;
- non modificare configurazioni Git globali/utente;
- non aggiungere file generati fuori scope;
- indicare ogni file modificato fuori dalla directory dello step;
- spiegare perché ogni modifica retroattiva è necessaria.

9.1 Baseline Git approvata per il live-critical set

Per i percorsi live, evitare il pinning SHA-256 manuale diffuso di manuali, report, contratti e test. La baseline revisionata deve essere identificata, quando applicabile, da un **commit SHA completo approvato esplicitamente dall'Utente/AI PM**. **Approvato** è uno stato di progetto, non una proprietà speciale di Git.

La verifica di integrità deve limitarsi al **live-critical set**, cioè ai file la cui modifica può cambiare direttamente ciò che viene inviato al sensore o i guardrail di esecuzione. Di norma comprende:

- launcher live corrente;
- backend USB reale;
- entripoint reale;
- moduli che possono inviare comandi USB o modificare il percorso live;
- eventuali guardrail software che implementano single-shot, cleanup/reseal o autorizzazione operatore.

Modifiche a manuale, report, manifest, test offline o altri artefatti non live-critical **non devono invalidare automaticamente** una baseline live già approvata.

Codex non deve inventare o auto-approvare il commit SHA. Se una run live richiede una baseline approvata e tale approvazione non esiste, la preparazione offline può continuare ma la run live resta non autorizzata.

Hash per-file indipendenti restano ammessi solo quando proteggono un artefatto esterno o un rischio concreto non coperto in modo equivalente dalla baseline Git; non devono essere usati come cerimonia generalizzata.

Se il repository non è interpretabile, fermarsi e riportare il problema all'Utente.

10. Sicurezza hardware e firmware

10.1 Operazioni vietate senza autorizzazione esplicita

Salvo task specifico autorizzato dall'Utente, l'AI esecutrice **non deve** eseguire:

- flashing, provisioning, OTP, IAP;
- modifica firmware del sensore;
- accesso al sensore USB reale;
- invio di comandi potenzialmente distruttivi;
- scrittura su memoria persistente del dispositivo;
- estrazione o manipolazione di chiavi;
- installazione di driver o plugin caricabili;

- attivazione di runtime libfprint;
- uso di `sudo` (per le AI — l'Utente sì, quando autorizza);
- operazioni di rete non necessarie.

10.2 Invarianti factory-preserving (requisiti, non cautele)

- Nessun erase o flash/IAP del firmware sensore;
- Nessun provisioning o rimpiazzo PSK;
- Nessuna scrittura OTP, factory data, configurazione persistente;
- Nessun cambio permanente di VID:PID o modalità boot;
- Nessuna procedura che renda incerta compatibilità Windows successiva;
- Nessun asset proprietario estratto da pacchetti OEM nei bundle pubblicabili.

10.3 Gerarchia di sicurezza (NUOVO)

La sicurezza del progetto si divide in due domini distinti, con priorità diverse:

Dominio	Priorità	Tipo di misure
Sicurezza dispositivo (hardware)	Massima	Factory-preserving, no-flash, no-OTP, no-PSK-replace
Sicurezza esecuzione (host)	Secondaria	Preflight, marker, sealing, dry-run

Regola operativa: Le misure di sicurezza host-side non devono mai diventare un ostacolo insuperabile all'esecuzione del codice quando il dispositivo è già al sicuro. Se un meccanismo host-side (es. marker single-use) blocca ripetutamente l'esecuzione senza proteggere realmente l'hardware, il meccanismo va **semplificato, archiviato manualmente o rimosso**, non ulteriormente stratificato.

11. Observability dei failure

Quando un gate fallisce, il sistema deve **esporre immediatamente la causa redatta** (non generica).

Esempi vietati:

- `root preflight failed` (senza causa);

- `unexpected_ack` (senza frame);
- `import failed` (senza modulo).

Esempi obbligatori:

```
PREFLIGHT_STATUS=fail
PREFLIGHT_FAILURES=single_use_marker
MARKER_PATH=/var/lib/goodix-5125-poc/d236-live-single-use.marker
USB_OPEN_COUNT=0
LIVE_USB_EXECUTION=NOT_STARTED
```

Un gate che produce report diagnostico ma lo nasconde all'operatore è un gate fallito.

12. Evidenze e linguaggio tecnico

Ogni affermazione importante deve essere classificata come:

- **osservato**: derivato direttamente da file, test, catture;
- **verificato**: confermato da procedura ripetibile;
- **inferito**: dedotto da più evidenze, non dimostrato direttamente;
- **ipotizzato**: spiegazione plausibile da verificare;
- **non noto**: informazione non disponibile.

13. Scope e milestone

Ogni milestone deve ridurre almeno una incertezza reale tra: protocollo, struttura firmware, inizializzazione, trasporto USB, cifratura, formato dati, interfaccia driver, integrazione libfprint, build, test, riproducibilità, sicurezza, manutenzione.

Non è una milestone valida:

- Produrre un bundle senza nuova evidenza device-side;
- Produrre report e review senza spostare il confine hardware;
- Risolvere difetti host-side che non erano prerequisiti della milestone.

14. Regole anti-frammentazione (NUOVO)

Per evitare il pattern D234-D239 (sei step per un unico avanzamento a E4 → A2#1):

1. **Fix di classe, non di opcode:** Se un fallimento appartiene a una classe (es. policy ACK troppo stretta), la correzione deve coprire l'intera classe prima di una nuova run live. No patch E4-only seguite da patch A2-only.
 2. **Audit orizzontale prima del live:** Prima di autorizzare una run live, tutte le fasi del protocollo atteso (E4, A2, 82, A6, A2, 70, 80×4, 90, D1, TLS) devono avere una policy di ACK/response completa ricostruita da transcript storici e capture.
 3. **Chiusura dei prerequisiti nello stesso step:** I difetti locali emersi mentre si costruisce un operator kit (shell, import, path, quoting, marker fixture) vanno chiusi **nello stesso step** prima della consegna. No D+1 per difetti locali di D.
 4. **Avanzamento roadmap ≠ numero D incrementato:** Avanzamento roadmap significa nuova evidenza tecnica o milestone reale, non numerazione di bundle.
 5. **Limite di 2 step consecutivi senza nuovo confine:** Se due step consecutivi non producono nuova evidenza device-side o nuovo comando raggiunto, l'AI PM deve fermarsi e riesaminare il metodo, non preparare un terzo step nello stesso schema.
-

15. Regole per prompt successivi

L'AI PM deve:

- Creare file `.md` scaricabile;
- Non limitarsi a mostrare il prompt in chat;
- Includere root del repository;
- Includere contesto tecnico sufficiente;
- Indicare obiettivo, attività, vincoli, criteri di accettazione;
- **Richiedere sempre Executable Closure Gate;**
- Richiedere sempre aggiornamento manuale;
- Richiedere sempre bundle step-local non cumulativo;
- Vietare espansioni di scope non autorizzate;

- Richiedere report finale trasparente.
-

16. Formato messaggio finale

Nel messaggio conclusivo, l'AI esecutrice deve includere:

- esito dello step;
 - decisione tecnica raggiunta;
 - file creati o modificati;
 - file modificati fuori directory dello step;
 - test eseguiti e esito;
 - **stato dell'Executable Closure Gate;**
 - limiti residui;
 - rischi o punti non verificati;
 - percorso del bundle ZIP;
 - conferma aggiornamento manuale tecnico;
 - eventuali azioni manuali richieste all'Utente.
-

17. Criterio generale di qualità

La qualità **non** si misura in:

- numero di file prodotti;
- numero di test passati offline;
- numero di report, bundle, artefatti;
- numero di step D-xxx generati.

La qualità si misura in:

- riduzione di un'incertezza reale;
- repository più comprensibile;
- progetto più riproducibile;
- assenza di rischi nascosti;
- documentazione di ciò che è noto e ciò che non lo è;

- avvicinamento concreto all'obiettivo finale;
 - **avanzamento del confine device-side del progetto.**
-

18. Stato attuale del progetto (snapshot revisione)

Per contesto operativo delle AI:

- **Massimo confine raggiunto live:** `E4 → A2 #1` con ACK `B0 / A2 / 07`
- **PSK/validator E4 binding:** `MATCH` confermato su target
- **Milestone atteso:** `FULL_PRE_D1_LIVE → D1 → TLS`
- **Milestone attuale:** `NOT YET EXECUTED`
- **Ultimo bundle prodotto:** D239 (74/74 test PASS, launcher reale fallito su `root preflight` con causa da leggere)
- **Blocco attuale:** Host-side (probabile marker storico D236 residuo) — risolvibile con comando manuale, non con nuovo bundle.

Direttiva immediata: Nessuna nuova run live autorizzata finché non viene risolto manualmente lo stato host-side bloccante e confermato con un launcher reale funzionante in ambiente operatore.

18. Revisione v2.1 — costituzione operativa, prompt delta e reporting minimo

Questa sezione **sostituisce ogni formulazione precedente incompatibile** relativa ad autosufficienza totale dei bundle, duplicazione delle regole permanenti nei prompt, pinning SHA-256 generalizzato e chiusure con campi ridondanti.

18.1 `AGENTS.md` come costituzione operativa di Codex

La root del repository deve contenere:

```
/home/guido/Repository/goodix-27c6-5125_private/AGENTS.md
```

`AGENTS.md` contiene solo le istruzioni permanenti che Codex deve conoscere in ogni sessione. Deve essere breve, stabile e mantenuto molto al di sotto del

limite di contesto dedicato ai project instructions. Non deve diventare una copia integrale delle presenti linee guida né del manuale tecnico.

Deve includere almeno:

- root canonica e manuale tecnico canonico;
- obbligo di leggere il manuale prima di modificare lo stato tecnico;
- invarianti factory-preserving e compatibilità Windows;
- divieti hardware permanenti salvo autorizzazione esplicita dell'Utente;
- disciplina Git e scope;
- Design → Implementazione → Esecuzione come ciclo massimo normale;
- definizione di avanzamento reale;
- riesame metodologico pre-live;
- obbligo di aggiornamento ricorsivo del manuale;
- bundle ZIP step-local non cumulativo;
- schema minimo di chiusura;
- distinzione tra safety telemetry e project reporting.

`AGENTS.md` è istruzione persistente per l'agente, **non enforcement software**. I guardrail che proteggono realmente hardware, secret o stato persistente devono restare implementati nel codice/launcher.

18.2 Prompt Codex: solo delta dello step

Il prompt specifico Dxxx non deve ricopiare le regole permanenti già presenti in `AGENTS.md`.

Deve contenere soltanto ciò che varia nello step:

- livello di ragionamento richiesto: `MEDIUM` o `HIGH`;
- contesto tecnico strettamente necessario;
- obiettivo specifico;
- scope e file particolari dello step;
- criteri di accettazione specifici;
- eventuali eccezioni esplicitamente autorizzate dall'Utente.

Il prompt non deve rilassare implicitamente i vincoli permanenti. Se uno step richiede una vera eccezione, questa deve essere esplicita, limitata e autorizzata dall'Utente. In caso di contraddizione non intenzionale tra task specifico e regole permanenti, Codex deve segnalarla invece di scegliere silenziosamente una reinterpretazione.

Le istruzioni dirette del task possono avere precedenza nel contesto operativo di Codex: per questo la prevenzione del conflitto deve avvenire **riducendo la duplicazione**, non affidandosi a una dichiarazione testuale secondo cui

`AGENTS.md` sarebbe tecnicamente insuperabile.

18.3 Riesame metodologico obbligatorio prima di ogni nuova run live

Prima di preparare o autorizzare una nuova run live dopo un fallimento precedente, Codex deve produrre un blocco visibile e conciso che risponda a tre domande:

1. **Cosa cambia realmente nel metodo rispetto all'ultimo tentativo?**
2. **Quale nuova ipotesi tecnica viene testata?**
3. **Se il tentativo fallisce di nuovo nello stesso punto, quale azione diversa verrà intrapresa?**

Il riesame deve distinguere una variazione sostanziale da modifiche cosmetiche a pacing, logging, packaging, hash, preflight o altre cautele host-side.

Se non esiste una risposta sostanziale alle prime due domande, non va creato un nuovo step live solo per ripetere lo stesso esperimento con nuova cerimonia.

Non creare per default un secondo diario canonico `method_log.md`: la conclusione metodologica rilevante deve confluire nel manuale tecnico canonico. Un log separato è ammesso solo se nasce una necessità concreta e viene esplicitamente autorizzato.

18.4 Numerazione Dxxx e correzioni locali

Un nuovo numero Dxxx è giustificato da almeno uno dei seguenti eventi:

- nuova esecuzione reale autorizzata;
- nuova evidenza tecnica;
- nuovo confine protocollo/hardware raggiunto;

- decisione architettureale che cambia realmente lo stato del progetto.

Una correzione locale dello stesso step, se non produce un nuovo confine tecnico, resta nello stesso Dxxx con revisione del relativo artefatto. La numerazione non deve misurare il volume di lavoro amministrativo.

18.5 Chiusura di progetto ridotta a sei campi

La sintesi finale di ogni step deve usare, salvo necessità concreta, soltanto:

1. `OUTCOME` — `READY | BLOCKED` + classificazione tecnica breve;
2. `ADVANCEMENT` — `REAL_EXECUTION_COMPLETED | NEW_TECHNICAL_EVIDENCE_PRODUCED | NONE` (combinazioni ammesse quando appropriate);
3. `EXECUTABLE_CLOSURE` — `PASS | FAIL | NOT_APPLICABLE` ;
4. `RESIDUAL_BLOCKER_OR_RISK` — descrizione sintetica;
5. `CANONICAL_DOCUMENTATION` — manuale aggiornato sì/no + sezioni toccate;
6. `BUNDLE` — path + SHA-256 del bundle step-local.

Campi aggiuntivi sono ammessi soltanto quando rappresentano un'informazione tecnica non derivabile e realmente utile alla decisione.

18.6 Safety telemetry ≠ project reporting

La riduzione dei campi di chiusura **non riduce la telemetria di sicurezza**.

I report macchina possono e devono conservare, quando pertinenti:

```
usb_open_count
command_count
tls_count
retry_count
D4/application_data reachability
claim/release
cleanup/reseal
device state
ACK/response osservati
persistent-write-family count
```

Questi dati servono a dimostrare il comportamento della run. Non devono però essere duplicati in più JSON, manifest e sintesi con nomi differenti se una singola fonte macchina è sufficiente.

18.7 Guardrail hardware da preservare

Non semplificare i guardrail economici che riducono direttamente un rischio sul device. In particolare, quando applicabili al percorso live, restano validi:

- verifica dell'operatore reale e del contesto privilegi (`root` / `SUDO_UID` o meccanismo equivalente coerente con il launcher);
- autorizzazione esplicita e single-shot per la run live;
- un solo tentativo live per autorizzazione/step;
- fail-closed sui comandi non compresi o potenzialmente persistenti;
- cleanup/release/reseal garantiti anche su uscita anomala;
- verifica di integrità del live-critical set contro la baseline Git approvata.

La semplificazione riguarda la duplicazione amministrativa, non la protezione del sensore.

18.8 Manuale tecnico come unico stato narrativo canonico

Il manuale tecnico root deve restare ordinato, leggibile e sufficiente a ricostruire nel tempo il percorso tecnico del progetto. Non deve trasformarsi in un log append-only dei bundle.

Quando una decisione cambia lo stato canonico:

- aggiornare le sezioni alte/canoniche interessate;
- aggiungere una sezione Dxxx solo se utile alla provenance;
- rimuovere o marcare chiaramente come superate le formulazioni non più correnti;
- non lasciare conoscenza nuova soltanto in chat, report o bundle.

18.9 Principio sintetico della v2.1

```
sicurezza hardware forte
+
contesto persistente breve
+
prompt specifici corti
+
una sola fonte narrativa canonica
+
telemetria macchina completa ma non duplicata
=
meno cerimonia senza ridurre l'ingegneria
```